

-: Data Structures :-

Introduction to Data Structures

A **data structure** is a systematic way of organizing, storing, and managing data in a computer so that it can be used efficiently. Every program in computer science involves handling data. Data structures provide a foundation for building efficient software applications.

- Data structures define how data is arranged in memory, how elements are connected, and what operations can be performed.
- The choice of data structure affects the **performance, scalability, and maintainability** of software.

Key Characteristics of Data Structures

1. **Organization of Data** – Determines how data is stored and linked.
 2. **Operations Supported** – Such as searching, insertion, deletion, traversal.
 3. **Memory Usage** – Efficiency of space utilization.
 4. **Time Efficiency** – Speed of accessing or modifying data.
 5. **Reusability** – A good data structure can be reused in different programs.
-

Classification of Data Structures

Data structures are broadly classified into:

1. **Primitive Data Structures**
 - Directly provided by programming languages.
 - Examples: Integer, Float, Character, Boolean.
 - Usually represent a single piece of information.
 2. **Non-Primitive Data Structures**
 - More complex and built using primitive types.
 - Classified as:
 - **Linear Data Structures** – Data elements arranged sequentially. (Examples: Arrays, Lists, Stacks, Queues).
 - **Non-Linear Data Structures** – Data elements connected in hierarchical or graph-based relationships. (Examples: Trees, Graphs).
-

Need for Data Structures

- To handle large volumes of data efficiently.
- To improve **time complexity** of operations like searching and sorting.
- To ensure **optimal memory usage**.

- To solve complex real-world problems like path finding, indexing, memory allocation, etc.
- To enhance **reusability and abstraction** in software.

Abstract Data Types (ADT)

An **Abstract Data Type (ADT)** is a **theoretical concept** that defines a data structure by its behavior (operations allowed) rather than its implementation.

- ADT = **Data + Operations**
 - It provides a logical description but hides implementation details.
1. **Encapsulation of Data and Operations**
 - ADT specifies what operations can be performed but not how they are implemented.
 2. **Mathematical Model**
 - ADTs are often described using algebraic or logical models.
 3. **Implementation Independence**
 - Same ADT can be implemented in multiple ways (e.g., a Queue can be implemented using arrays or linked lists).

Examples of ADTs

- **List ADT**: Operations → Insert, Delete, Traverse.
- **Stack ADT**: Operations → Push, Pop, Peek.
- **Queue ADT**: Operations → Enqueue, Dequeue.
- **Graph ADT**: Operations → Add Vertex, Add Edge, Traversal.

Advantages of ADT

- Enhances **abstraction** (users need not know internal details).
- Promotes **modularity** (independent implementation).
- Encourages **code reusability**.
- Provides a **standardized model** for problem solving.

Complexity of Algorithms

When analyzing algorithms, we focus on **time complexity** and **space complexity**.

1. Time Complexity

Time complexity measures how the **execution time** of an algorithm grows with the size of input (n).

- Expressed using **Big O notation**.
- Examples:
 - Constant Time: $O(1)$

- Linear Time: $O(n)$
- Logarithmic Time: $O(\log n)$
- Quadratic Time: $O(n^2)$

2. Space Complexity

Space complexity measures the amount of memory required by an algorithm during execution.

- Includes memory for:
 1. Input data
 2. Auxiliary variables
 3. Recursion stack (if any)

3. Common Cases in Complexity

- **Best Case** – Minimum time taken for smallest/ideal input.
 - **Worst Case** – Maximum time taken for the worst possible input.
 - **Average Case** – Expected time for random input distribution.
-

Lists

A **List** is a linear collection of elements arranged in a sequence.

Operations on Lists

- Insertion
- Deletion
- Traversal
- Searching
- Updating

Types of Lists

1. **Array-based List** – Stored in contiguous memory.
 2. **Linked List** – Elements connected using pointers.
-

Stack

A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle.

Basic Operations

1. **Push** – Insert element at the top.
2. **Pop** – Remove element from the top.

3. **Peek/Top** – Retrieve top element without removing.

Applications

- Expression evaluation
 - Undo functionality
 - Function call management
-

Queue

A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle.

Basic Operations

1. **Enqueue** – Insert element at the rear.
2. **Dequeue** – Remove element from the front.
3. **Front/Peek** – Access front element.

Types of Queues

- **Simple Queue** – Basic FIFO.
 - **Circular Queue** – Last position connected to first.
 - **Priority Queue** – Elements served by priority.
 - **Deque (Double-ended Queue)** – Insertion/deletion allowed at both ends.
-

Introduction to Sorting

Sorting is the process of **arranging elements** of a collection in a specific order (ascending or descending).

It is one of the most common and important operations in computer science because:

- Sorted data makes searching easier.
- Sorting helps in efficient storage and retrieval.
- Many real-world applications (databases, file systems) depend on sorting.

Sorting algorithms differ in **time complexity, space complexity, and stability**.

Selection Sort

- Repeatedly find the smallest element from the unsorted portion and place it at the beginning.

Steps

1. Start from the first element.
2. Find the smallest element in the array.
3. Swap it with the first element.
4. Repeat for the remaining (n-1) elements.

Pseudo-code

```
for i = 0 to n-2:
    min_index = i
    for j = i+1 to n-1:
        if A[j] < A[min_index]:
            min_index = j
    swap A[i], A[min_index]
```

Complexity

- Best Case: $O(n^2)$
 - Worst Case: $O(n^2)$
 - Space: $O(1)$
-

Insertion Sort

- Build the sorted list one element at a time by inserting each new element into its correct position.

Steps

1. Start with the second element.
2. Compare it with elements before it.
3. Shift larger elements one step to the right.
4. Insert the current element in its correct place.

Complexity

- Best Case: $O(n)$ (already sorted).
 - Worst Case: $O(n^2)$.
 - Space: $O(1)$.
-

Bubble Sort

- Repeatedly swap adjacent elements if they are in the wrong order.

Steps

1. Compare each pair of adjacent elements.
2. Swap them if needed.
3. After first pass, largest element moves to last position.
4. Repeat for remaining elements.

Complexity

- Best Case: $O(n)$.
 - Worst Case: $O(n^2)$.
 - Space: $O(1)$.
-

Quick Sort

- A **divide-and-conquer** algorithm.
- Choose a pivot element, partition the array into two sub-arrays, then recursively sort them.

Steps

1. Select a pivot.
2. Rearrange elements so that all elements smaller than pivot are left, and greater are right.
3. Recursively apply the same to sub-arrays.

Complexity

- Best Case: $O(n \log n)$.
 - Average Case: $O(n \log n)$.
 - Worst Case: $O(n^2)$.
 - Space: $O(\log n)$ (recursion).
-

Merge Sort

- Another **divide-and-conquer** algorithm.
- Divide the array into halves, sort each half, and merge them.

Steps

1. Split array into two halves.
2. Recursively sort both halves.
3. Merge sorted halves into one sorted array.

Complexity

- Best Case: $O(n \log n)$.
 - Worst Case: $O(n \log n)$.
 - Space: $O(n)$.
-

Heap Sort

- Uses a **binary heap** data structure.
- Build a max-heap and repeatedly extract the largest element.

Steps

1. Convert array into max-heap.
2. Swap the root (largest) with last element.
3. Reduce heap size by one and heapify.
4. Repeat until array is sorted.

Complexity

- Best Case: $O(n \log n)$.
 - Worst Case: $O(n \log n)$.
 - Space: $O(1)$.
-

Introduction to Searching

Searching means finding whether a given element exists in a collection, and if it does, retrieving its position.

Sequential (Linear) Search

- Compare each element sequentially until the desired element is found or list ends.

Steps

1. Start at first element.
2. Compare with key.
3. If found, return position.
4. Otherwise, continue.

Complexity

- Best Case: $O(1)$.
 - Worst Case: $O(n)$.
-

Binary Search

- Works only on **sorted arrays**.
- Repeatedly divide search space in half.

Steps

1. Compare middle element with key.
2. If equal, return index.
3. If $\text{key} < \text{middle}$, search left half.
4. Else, search right half.

Complexity

- Best Case: $O(1)$.
 - Worst Case: $O(\log n)$.
-

Hashing

- Uses a **hash function** to directly compute storage location of an element.

Steps

1. Compute index using hash function.
2. Store element at computed index.
3. Handle collisions using methods like chaining or open addressing.

Complexity

- Best Case: $O(1)$.
 - Worst Case: $O(n)$ (if collisions).
-

Introduction to Trees

A **tree** is a hierarchical data structure consisting of **nodes** connected by **edges**. Unlike linear data structures (arrays, lists), trees represent data in a **parent-child relationship**.

- **Root Node** – The topmost node in a tree. Only one root exists.
- **Parent Node** – Any node with child nodes.
- **Child Node** – Nodes connected below a parent.
- **Leaf Node** – Nodes with no children.
- **Sibling Nodes** – Nodes sharing the same parent.
- **Degree of a Node** – Number of children a node has.
- **Depth / Level** – Distance from root (root level = 0).
- **Height of a Tree** – Maximum depth among all nodes.

Trees are widely used to represent hierarchical relationships, such as file directories or organization charts.

Types of Trees

General Tree

- A node can have any number of children.
- Flexible but harder to implement.

Binary Tree

- Each node can have **at most two children** (left and right).
- Used in many algorithms like sorting, searching, and parsing expressions.

Binary Search Tree (BST)

- A binary tree with a **key ordering property**:
 - Left child < Parent node
 - Right child > Parent node
 - Allows **efficient searching, insertion, and deletion** operations.
-

Operations on Binary Trees

1. **Insertion**
 - Add a new node at the correct position maintaining BST property.
2. **Deletion**
 - Remove a node while maintaining BST properties.
 - Cases:
 - Node has no child → delete directly.
 - Node has one child → link child to parent.

- Node has two children → replace with inorder predecessor/successor.
3. **Traversal**
- Visit every node **exactly once**.
-

Tree Traversal Methods

1. Depth-First Traversal

- Explore as deep as possible before backtracking.
- Types:
 1. **Inorder (Left → Root → Right)** – Produces sorted order for BST.
 2. **Preorder (Root → Left → Right)** – Useful for copying trees.
 3. **Postorder (Left → Right → Root)** – Useful for deleting nodes.

2. Breadth-First Traversal (Level Order)

- Visit nodes **level by level** from top to bottom.
 - Typically implemented using a **queue**.
-

3. Properties of BST

- Searching, insertion, and deletion operations are **$O(\log n)$** on average.
 - Performance depends on **tree balance** (unbalanced tree → worst case $O(n)$).
-

Introduction to Graphs

A **graph** is a **non-linear data structure** consisting of:

- **Vertices (nodes)** – Represent entities.
- **Edges (connections)** – Represent relationships between entities.

Graphs are widely used in:

- Network routing
 - Social networks
 - Scheduling problems
 - Pathfinding algorithms
-

Types of Graphs

1. Based on Edges

- Undirected Graph**
 - Edges have no direction.
 - Example: Friendships in social networks.
- Directed Graph (Digraph)**
 - Edges have a direction.
 - Example: One-way streets in maps.
- Weighted Graph**
 - Edges have weights (cost, distance, time).
 - Example: Road network with distances.
- Unweighted Graph**
 - All edges have the same value or no weight.

2. Based on Connectivity

- Connected Graph**
 - There is a path between every pair of vertices.
 - Disconnected Graph**
 - Some vertices are not reachable from others.
-

Graph Representation

1. Adjacency Matrix

- A **2D array** of size $V \times V$, where V = number of vertices.
- $\text{matrix}[i][j] = 1$ if there is an edge between vertex i and j .
- Simple but consumes **$O(V^2)$ space**.

2. Adjacency List

- Each vertex has a **list of connected vertices**.
 - Efficient for **sparse graphs**.
-

Graph Traversal Techniques

1. Depth-First Search (DFS)

- Explores as far as possible along each branch before backtracking.
- Can be implemented using **recursion** or **stack**.

DFS Steps:

1. Start at a selected vertex.
2. Mark it as visited.
3. Visit an adjacent unvisited vertex recursively.
4. Repeat until all vertices are visited.

2. Breadth-First Search (BFS)

- Visits all vertices **level by level**.
- Implemented using a **queue**.

BFS Steps:

1. Start at a selected vertex.
 2. Enqueue it and mark as visited.
 3. Dequeue vertex and visit its unvisited adjacent vertices.
 4. Repeat until queue is empty.
-

Properties of Graphs

- **Degree of Vertex:** Number of edges incident on a vertex.
 - **In-degree / Out-degree:** For directed graphs, incoming and outgoing edges.
 - **Connectivity:** Determines whether vertices are reachable.
 - **Cycles:** Path where starting and ending vertices are the same.
-

Applications of Graphs

- Finding shortest paths (Dijkstra's algorithm).
- Network routing and flow analysis.

- Dependency resolution (e.g., compilation order).
- Modeling real-world structures like maps and circuits.

